Deep Learning

Mihir Rao

Contents

1 Introduction **2**
1.1 Translation and shared representations 2
1.2 Learning a function and its features 2
1.3 Artificial neurons 3
1.4 Capacity, inductive bias, and scale 4

Lecture 1: Introduction

1.1 Translation and shared representations

A translation system learns from paired data: examples of the same content in two languages. A separate model for each language pair depends on having suitable examples for that pair. This becomes difficult when data for a particular pair are scarce.

A multilingual model shares parameters across language pairs and receives a requested target language. Sharing lets training data for one pair inform another. In the illustrated setup, the model is trained on English to French, French to English, and English to Spanish, then asked to translate French to Spanish without examples in that direction. This is zero-shot translation: the requested direction is absent from training, even though its languages occur in other training pairs.

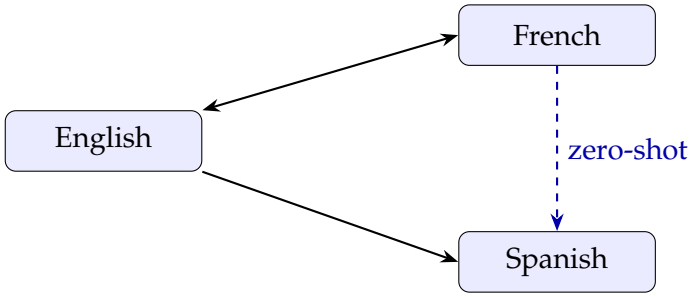


Figure 1: Solid arrows are training directions; the dashed direction is tested.

Interpolating between target-language signals can produce outputs that blend properties of the requested languages, though the result need not be a valid sentence in either language. These behaviors suggest that the model has learned useful internal structure shared across tasks; they do not prove that it has discovered a unique, human-like representation of meaning.

1.2 Learning a function and its features

When an input-output rule is difficult to write explicitly, machine learning fits a parameterized function to examples. For a simple two-feature classifier, parameters can describe a line separating predicted classes. The important design choice is the parameterization: which functions can be expressed and how the parameters determine them.

Raw images or sentences are much more complex than two numerical features. One approach constructs features by hand and learns a prediction rule on top of them. This moves the hard design problem into choosing the features. In REPRESENTATION LEARNING, the features themselves are learned from data; DEEP LEARNING uses multiple layers of learned representations. Each layer transforms the previous layer, producing a sequence such as

$$h_0 = x, \qquad h_1 = f_1(h_0; \theta_1), \quad h_2 = f_2(h_1; \theta_2), \quad \widehat{y} = f_3(h_2; \theta_3).$$

Here x is the input, h_1, h_2 are intermediate representations, $\widehat{y}$ is the prediction, and θ_i are learned parameters.

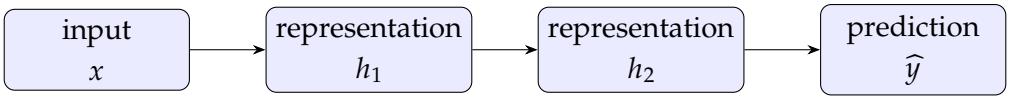


Figure 2: A deep model composes learned transformations.

Later representations can capture more abstract, task-relevant properties and become less sensitive to nuisance variation. This is a useful aim, not a guarantee for every learned layer. In END-TO-END LEARNING, parameters throughout the model are usually trained against a common task objective, so the representation can adapt to the prediction task. Learning all layers jointly is common, but not required in every deep-learning system.

1.3 Artificial neurons

An artificial unit combines inputs using weights, adds a bias, and applies an activation function. If $x_1, \ldots, x_d$ are its inputs, its output is

$$z = \sum_{i=1}^{d} w_i x_i + b, \qquad h = \sigma(z).$$

The weights w_i and bias b are parameters; σ determines how the weighted sum becomes an activation passed to the next layer.

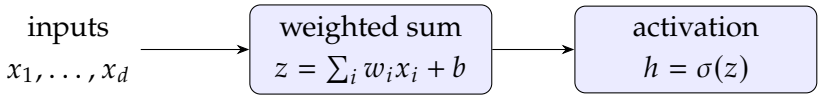


Figure 3: A unit transforms its inputs into an activation.

The name *neural network* reflects an early analogy to biological neurons receiving and transmitting signals. Artificial units are greatly simplified, so the analogy does not establish that brains use the same learning procedure.

1.4 Capacity, inductive bias, and scale

Model capacity describes, informally, the range of functions a model family can represent. Inductive bias is what is built into a learning system that makes some solutions more likely than others. Learning features reduces the need to specify each feature by hand, but the model, training procedure, and data still encode choices and assumptions.

Much of deep learning's progress has depended on the combination of large models, large datasets, and sufficient computation. A method *scales* when additional data, model capacity, and compute can improve its results. These resources have costs, and more of them does not by itself guarantee a better model. The historical examples range from the perceptron and backpropagation to LeNet and the 2012 AlexNet result, illustrating that the ideas and the resources needed to apply them developed over time.

Source. Sergey Levine, Deep Learning, Spring 2021, Lecture 1.